

Assignment 5

Kaggle Competition and Unsupervised Learning

Aesha Gandhi 2026-03-26

Imports/Setup

```
In [4]: # ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

```
In [ ]: # # mount drive
# from google.colab import drive
# drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [3]: !pip install -q jupyter_black
import jupyter_black

jupyter_black.load()
jupyter_black.load(line_length=79)

# <div style="page-break-before: always;"></div>
```

```
In [5]: # installs
!pip install xgboost lightgbm optuna -q
```

0.0/419.5 kB ? eta -:--:--

389.1/419.5 kB 11.6 MB/s eta 0:00

0:01

419.5/419.5 kB 8.3 MB/s eta 0:00

0:00

```
In [6]: # imports
import pandas as pd
import numpy as np
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    roc_auc_score,
    average_precision_score,
    roc_curve,
    precision_recall_curve
)
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from xgboost import XGBClassifier
import lightgbm as lgb
import optuna
from xgboost import XGBClassifier
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression as LR

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.ensemble import RandomForestClassifier,  
GradientBoostingClassifier
```

Exercise 1: Kaggle

1.2: Preprocess Data

```
In [ ]: #####
# Load the data
#####
data = pd.read_pickle("/content/drive/MyDrive/Duke-
MIDS/ML/HW5/data/a5_q1.pkl")

y_train = data['y_train']
X_train_original = data['X_train'] # Original dataset
X_train_ohe = data['X_train_ohe'] # One-hot-encoded dataset

X_test_original = data['X_test']
X_test_ohe = data['X_test_ohe']

#####
# Produce submission
#####

def create_submission(confidence_scores, save_path):
    '''Creates an output file of submissions for Kaggle

    Parameters
    -----
    confidence_scores : list or numpy array
        Confidence scores (from predict_proba methods from classifiers) or
        binary predictions (only recommended in cases when predict_proba is
        not available)
    save_path : string
        File path for where to save the submission file.

    Example:
    create_submission(my_confidence_scores, './data/submission.csv')

    ...

    submission = pd.DataFrame({"score":confidence_scores})
    submission.to_csv(save_path, index_label="id")
```

```
In [ ]: # feature engineering - add total guests, total nights, total costs,
        booking changes, is family?
def add_features(df):
    df = df.copy()

    df["total_guests"] = (
        df["adults"] +
        df["children"].fillna(0) +
        df["babies"]
    )

    df["total_nights"] = (
        df["stays_in_weekend_nights"] +
        df["stays_in_week_nights"]
    )

    df["total_cost"] = df["adr"] * df["total_nights"]

    df["booking_changes_ratio"] = (
        df["booking_changes"] / (df["lead_time"] + 1)
    )

    df["is_family"] = (
        (df["children"].fillna(0) + df["babies"]) > 0
    ).astype(int)

    df = df.drop(columns=["agent", "company"], errors="ignore")
```

```

    return df

# data preprocessing
# one hot encode features

X_train = add_features(X_train_ohe).copy()
X_test = add_features(X_test_ohe).copy()
y_train = y_train.copy()

# inspect data
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)

print("\nMissing values in training data:")
print(X_train.isnull().sum().sort_values(ascending=False).head(10))

print("\nMissing values in test data:")
print(X_test.isnull().sum().sort_values(ascending=False).head(10))

# check if duplicated rows in training data
print("\nDuplicate rows in X_train:", X_train.duplicated().sum())

# split data into training and validation
X_train_split, X_val, y_train_split, y_val = train_test_split(
    X_train,
    y_train,
    test_size=0.2,
    random_state=42,
    stratify=y_train
)

print("\nAfter validation split:")
print("X_train_split:", X_train_split.shape)
print("X_val:", X_val.shape)

# look at missing values and use median imputation

imputer = SimpleImputer(strategy="median")

X_train_imputed = pd.DataFrame(
    imputer.fit_transform(X_train_split),
    columns=X_train_split.columns,
    index=X_train_split.index
)

X_val_imputed = pd.DataFrame(
    imputer.transform(X_val),
    columns=X_val.columns,
    index=X_val.index
)

X_test_imputed = pd.DataFrame(
    imputer.transform(X_test),
    columns=X_test.columns,
    index=X_test.index
)

# normalize and standardize features
scaler = StandardScaler()

X_train_scaled = pd.DataFrame(
    scaler.fit_transform(X_train_imputed),
    columns=X_train_imputed.columns,
    index=X_train_imputed.index
)

X_val_scaled = pd.DataFrame(
    scaler.transform(X_val_imputed),
    columns=X_val_imputed.columns,
    index=X_val_imputed.index
)

```

```

X_test_scaled = pd.DataFrame(
    scaler.transform(X_test_imputed),
    columns=X_test_imputed.columns,
    index=X_test_imputed.index
)

print("\nAny missing values left?")
print("Train:", X_train_scaled.isnull().sum().sum())
print("Val:", X_val_scaled.isnull().sum().sum())
print("Test:", X_test_scaled.isnull().sum().sum())

print("\nFinished preprocessing.")
print("Scaled train shape:", X_train_scaled.shape)
print("Scaled val shape:", X_val_scaled.shape)
print("Scaled test shape:", X_test_scaled.shape)

```

```

X_train shape: (95512, 945)
X_test shape: (23878, 945)
y_train shape: (95512,)

```

```

Missing values in training data:
children          2
company_67.0      0
company_68.0      0
company_71.0      0
company_72.0      0
company_73.0      0
company_76.0      0
company_77.0      0
company_78.0      0
company_80.0      0
dtype: int64

```

```

Missing values in test data:
children          2
company_67.0      0
company_68.0      0
company_71.0      0
company_72.0      0
company_73.0      0
company_76.0      0
company_77.0      0
company_78.0      0
company_80.0      0
dtype: int64

```

```

Duplicate rows in X_train: 24648

```

```

After validation split:
X_train_split: (76409, 945)
X_val: (19103, 945)

```

```

Any missing values left?
Train: 0
Val: 0
Test: 0

```

```

Finished preprocessing.
Scaled train shape: (76409, 945)
Scaled val shape: (19103, 945)
Scaled test shape: (23878, 945)

```

In preprocessing, I first did feature engineering to add more informative variables: total guests, total nights, total cost to capture more information about booking. I added booking changes ratio to normalize the changes by lead time, and an indicator for whether it is a family/household. These features could help the model learn more patterns. I dropped the features agent and company because those seem like they could be adding more noise than useful.

The data was already provided as one hot encoded which was important for transforming categorical data into features we can use for models. The data is split into training and validation sets with stratification so that the class distribution is preserved. I handle missing values using median imputation since that is robust to outliers and can prevent data leakage by fitting only on training data and applying that to validation/test sets. I use standardization to make sure the features are on the same scale, which is important for models sensitive to feature magnitude (such as logistic regression, SVMs, KNN).

1.3: Train and Compare Models

```
In [ ]: # store model results
results = []

def evaluate_model(name, model, X_tr, y_tr, X_v, y_v):
    # training time
    train_start = time.perf_counter()
    model.fit(X_tr, y_tr)
    train_end = time.perf_counter()
    train_time = train_end - train_start

    # inference time
    pred_start = time.perf_counter()
    y_val_pred = model.predict(X_v)

    if hasattr(model, "predict_proba"):
        y_val_score = model.predict_proba(X_v)[:, 1]
    elif hasattr(model, "decision_function"):
        y_val_score = model.decision_function(X_v)
    else:
        y_val_score = y_val_pred

    pred_end = time.perf_counter()
    pred_time = pred_end - pred_start

    # evaluation metrics
    acc = accuracy_score(y_v, y_val_pred)
    auc = roc_auc_score(y_v, y_val_score)
    ap = average_precision_score(y_v, y_val_score)

    # roc curve points
    fpr, tpr, _ = roc_curve(y_v, y_val_score)

    # precision recall curve points
    precision, recall, _ = precision_recall_curve(y_v, y_val_score)

    # save results
    results.append({
        "Model": name,
        "Validation Accuracy": acc,
        "Validation AUC": auc,
        "Validation AP": ap,
        "Training Time (s)": train_time,
        "Prediction Time (s)": pred_time,
        "fpr": fpr,
        "tpr": tpr,
        "precision": precision,
        "recall": recall
    })

    # print summary
    print(f"{name}")
    print(f"  Accuracy:          {acc:.4f}")
    print(f"  ROC AUC:           {auc:.4f}")
    print(f"  Average Precision: {ap:.4f}")
    print(f"  Training Time:     {train_time:.4f} s")
    print(f"  Prediction Time:   {pred_time:.4f} s\n")

    return model

def plot_roc_curves(results):
    plt.figure(figsize=(8, 6))

    for res in results:
        plt.plot(
            res["fpr"],
            res["tpr"],
            label=f'{res["Model"]} (AUC = {res["Validation AUC"]:.3f})'
```



```

    )

    # line to show guessing at random
    plt.plot([0, 1], [0, 1], linestyle="--", label="Random Guessing")

    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curves on Validation Data")
    plt.legend()
    plt.grid(True)
    plt.show()

def plot_pr_curves(results, y_v):
    plt.figure(figsize=(8, 6))

    for res in results:
        plt.plot(
            res["recall"],
            res["precision"],
            label=f'{res["Model"]} (AP = {res["Validation AP"]:.3f})'
        )

    # line to show guessing at random
    baseline = np.mean(y_v)
    plt.axhline(y=baseline, linestyle="--", label=f"Random Guessing ({baseline:.3f})")

    plt.xlabel("Recall")
    plt.ylabel("Precision")
    plt.title("Precision-Recall Curves on Validation Data")
    plt.legend()
    plt.grid(True)
    plt.show()

def plot_model_times(results):
    results_df = pd.DataFrame(results)

    plot_df = results_df[["Model", "Training Time (s)", "Prediction Time (s)"]].copy()
    plot_df = plot_df.sort_values("Training Time (s)", ascending=False)

    x = np.arange(len(plot_df))
    width = 0.35

    plt.figure(figsize=(10, 6))
    plt.bar(x - width/2, plot_df["Training Time (s)"], width,
            label="Training Time")
    plt.bar(x + width/2, plot_df["Prediction Time (s)"], width,
            label="Prediction Time")

    plt.xticks(x, plot_df["Model"], rotation=45, ha="right")
    plt.ylabel("Time (seconds)")
    plt.title("Training and Prediction Time by Model")
    plt.legend()
    plt.grid(axis="y")
    plt.tight_layout()
    plt.show()

def results_table(results):
    df = pd.DataFrame(results)[[
        "Model",
        "Validation Accuracy",
        "Validation AUC",
        "Validation AP",
        "Training Time (s)",
        "Prediction Time (s)"
    ]].copy()

```

```
return df.sort_values("Validation AUC",
ascending=False).reset_index(drop=True)
```

```
In [ ]: log_reg = LogisticRegression(max_iter = 2000, random_state=42)
log_reg = evaluate_model("Logistic Regression", log_reg, X_train_scaled,
y_train_split, X_val_scaled, y_val)
```

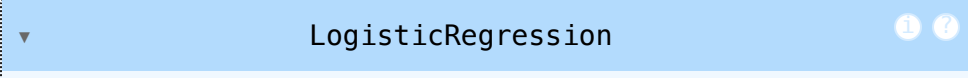
Logistic Regression

Accuracy:	0.8300
ROC AUC:	0.9102
Average Precision:	0.8680
Training Time:	9.9834 s
Prediction Time:	0.0722 s

```
In [ ]: # logistic regression with regularization
log_reg_l2_stronger = LogisticRegression(C=0.1, max_iter=2000,
random_state=42)
evaluate_model(
    "Logistic Regression (C=0.1)",
    log_reg_l2_stronger,
    X_train_scaled, y_train_split,
    X_val_scaled, y_val
)
```

Logistic Regression (C=0.1)

Accuracy:	0.8295
ROC AUC:	0.9102
Average Precision:	0.8688
Training Time:	6.5329 s
Prediction Time:	0.1285 s

Out []: 
LogisticRegression(C=0.1, max_iter=2000, random_state=42)

```
In [ ]: knn = KNeighborsClassifier(n_neighbors = 15)
knn = evaluate_model("KNN", knn, X_train_scaled, y_train_split,
X_val_scaled, y_val)
```

KNN

Accuracy:	0.8071
ROC AUC:	0.8810
Average Precision:	0.8326
Training Time:	2.2933 s
Prediction Time:	206.5588 s

```
In [ ]: rf = RandomForestClassifier(
    n_estimators=300,
    max_depth=12,
    min_samples_split=10,
    random_state=42,
    n_jobs=-1
)
rf = evaluate_model(
    "Random Forest",
    rf,
    X_train_imputed, y_train_split,
    X_val_imputed, y_val
)
```

Random Forest

Accuracy:	0.7768
ROC AUC:	0.9172
Average Precision:	0.8899
Training Time:	38.0806 s
Prediction Time:	1.1389 s

```
In [ ]: gb = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
```

```

        max_depth=3,
        subsample=0.8,
        random_state=42
    )
    gb = evaluate_model(
        "Gradient Boost",
        gb,
        X_train_imputed, y_train_split,
        X_val_imputed, y_val
    )

```

Gradient Boost

```

Accuracy:      0.8532
ROC AUC:       0.9282
Average Precision: 0.8970
Training Time: 186.1952 s
Prediction Time: 0.5093 s

```

In []:

Complex Models: XGBoost, LightGBM (Optuna), and Stacking Ensemble

In []:

```

# tuned xgboost

# use early stopping to find the best n_estimators
xgb_probe = XGBClassifier(
    n_estimators=1000,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=3,
    gamma=0.1,
    reg_alpha=0.1,
    reg_lambda=1.0,
    use_label_encoder=False,
    eval_metric="auc",
    early_stopping_rounds=50,
    random_state=42,
    n_jobs=-1,
    verbosity=0,
)
xgb_probe.fit(
    X_train_imputed, y_train_split,
    eval_set=[(X_val_imputed, y_val)],
    verbose=False,
)
best_n = xgb_probe.best_iteration
print(f"Best n_estimators from early stopping: {best_n}")

# now rebuild with fixed n_estimators
xgb_model = XGBClassifier(
    n_estimators=best_n,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    min_child_weight=3,
    gamma=0.1,
    reg_alpha=0.1,
    reg_lambda=1.0,
    use_label_encoder=False,
    eval_metric="auc",
    random_state=42,
    n_jobs=-1,
    verbosity=0,
)

xgb_model = evaluate_model(
    "XGBoost (tuned)",

```

```

    xgb_model,
    X_train_imputed, y_train_split,
    X_val_imputed, y_val,
)

```

Best n_estimators from early stopping: 999

XGBoost (tuned)

```

Accuracy:      0.8805
ROC AUC:       0.9536
Average Precision: 0.9316
Training Time: 140.6517 s
Prediction Time: 2.2766 s

```

In []: *# lightGBM with Optuna*

```

optuna.logging.set_verbosity(optuna.logging.WARNING)

def lgbm_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 300, 1200),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.15,
log=True),
        "num_leaves": trial.suggest_int("num_leaves", 31, 255),
        "max_depth": trial.suggest_int("max_depth", 4, 10),
        "min_child_samples": trial.suggest_int("min_child_samples", 10,
100),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6,
1.0),
        "reg_alpha": trial.suggest_float("reg_alpha", 1e-4, 10.0,
log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 1e-4, 10.0,
log=True),
        "random_state": 42,
        "n_jobs": -1,
        "verbosity": -1,
    }
    clf = lgb.LGBMClassifier(**params)
    clf.fit(
        X_train_imputed, y_train_split,
        eval_set=[(X_val_imputed, y_val)],
    )
    val_probs = clf.predict_proba(X_val_imputed)[: , 1]
    return roc_auc_score(y_val, val_probs)

study = optuna.create_study(direction="maximize")
study.optimize(lgbm_objective, n_trials=40, show_progress_bar=True)

print("Best LightGBM CV AUC:", study.best_value)
print("Best params:", study.best_params)

best_lgbm = lgb.LGBMClassifier(
    **study.best_params,
    random_state=42,
    n_jobs=-1,
    verbosity=-1,
)

best_lgbm = evaluate_model(
    "LightGBM (Optuna)",
    best_lgbm,
    X_train_imputed, y_train_split,
    X_val_imputed, y_val,
)

0%|          | 0/40 [00:00<?, ?it/s]

```

Best LightGBM CV AUC: 0.9582064891417816

Best params: {'n_estimators': 926, 'learning_rate': 0.11744322000879612, 'num_leaves': 108, 'max_depth': 10, 'min_child_samples': 25, 'subsample': 0.8064911579603443, 'colsample_bytree': 0.9578924767923324, 'reg_alpha': 0.007986565406606692, 'reg_lambda': 0.49581676478526865}

LightGBM (Optuna)

Accuracy: 0.8894
ROC AUC: 0.9582
Average Precision: 0.9391
Training Time: 13.2533 s
Prediction Time: 6.4781 s

```
In [ ]: # stacking ensemble: XGB + LightGBM + RF
```

```
estimators = [  
    ("xgb", XGBClassifier(  
        n_estimators=800, learning_rate=0.05, max_depth=6,  
        subsample=0.8, colsample_bytree=0.8, min_child_weight=3,  
        gamma=0.1, reg_alpha=0.1, reg_lambda=1.0,  
        use_label_encoder=False, eval_metric="auc",  
        random_state=42, n_jobs=-1, verbosity=0,  
    )),  
    ("lgbm", lgb.LGBMClassifier(  
        **study.best_params, random_state=42, n_jobs=-1, verbosity=-1,  
    )),  
    ("rf", RandomForestClassifier(  
        n_estimators=300, max_depth=12, min_samples_split=10,  
        random_state=42, n_jobs=-1,  
    )),  
]  
  
stack_clf = StackingClassifier(  
    estimators=estimators,  
    final_estimator=LR(C=1.0, max_iter=1000),  
    cv=5,  
    stack_method="predict_proba",  
    n_jobs=-1,  
    passthrough=False,  
)  
  
stack_clf = evaluate_model(  
    "Stacking (XGB+LGBM+RF)",  
    stack_clf,  
    X_train_imputed, y_train_split,  
    X_val_imputed, y_val,  
)
```

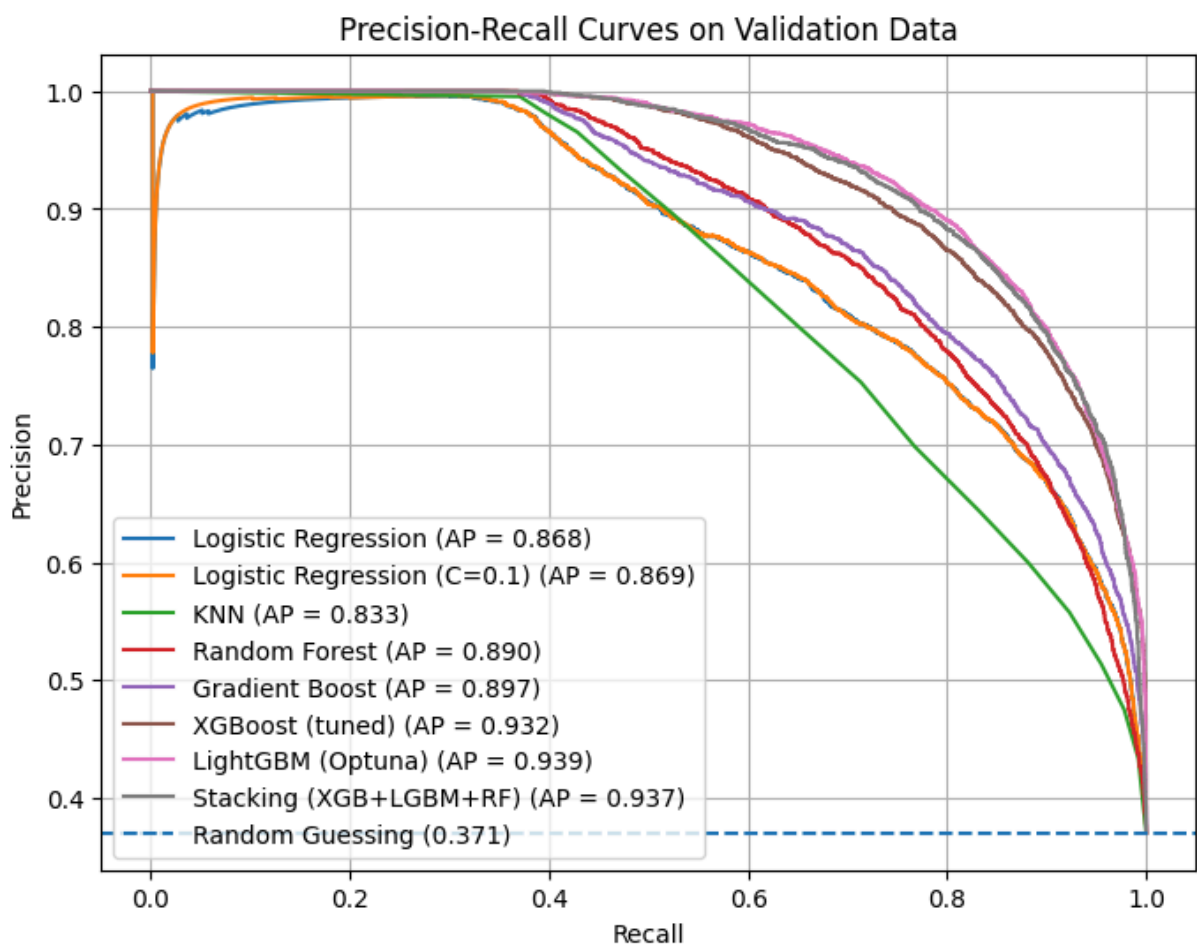
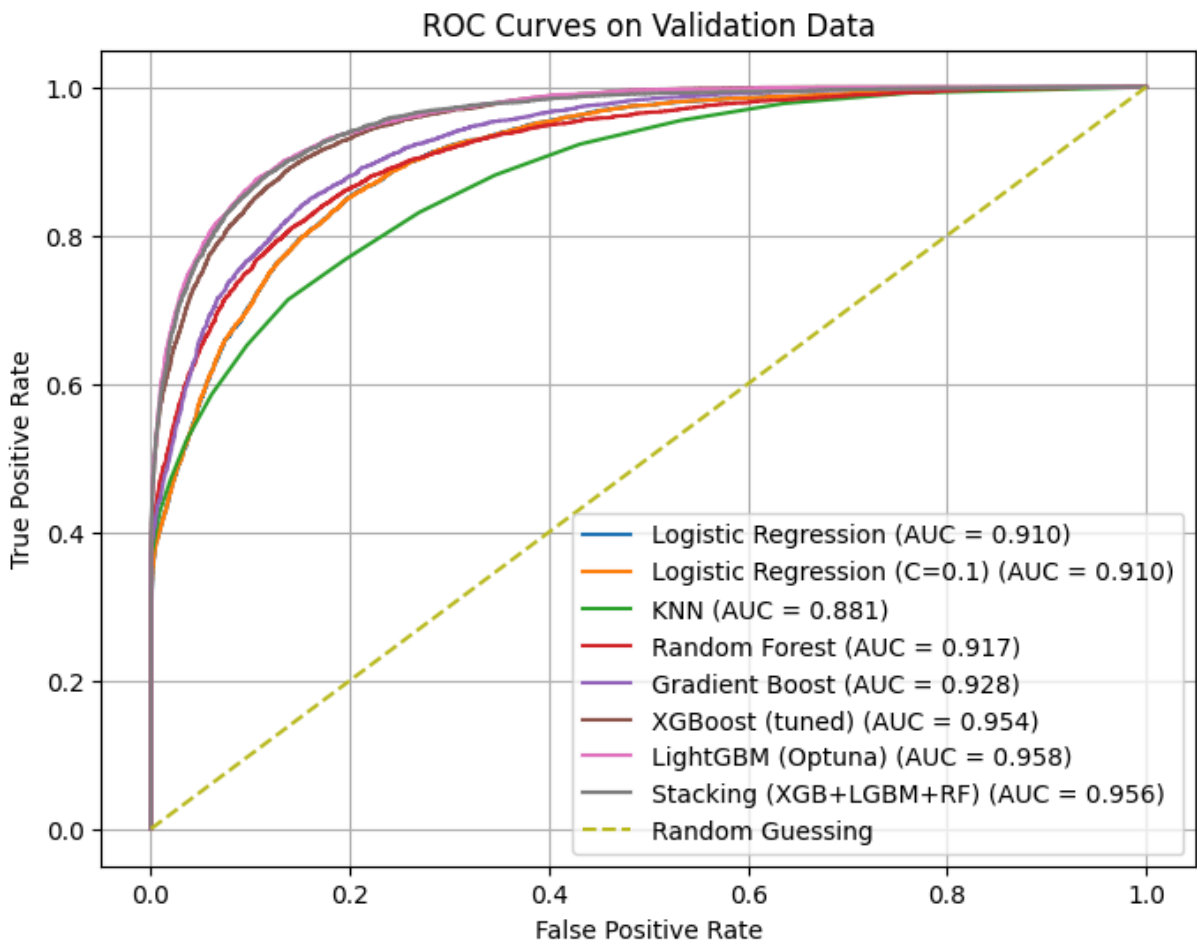
Stacking (XGB+LGBM+RF)

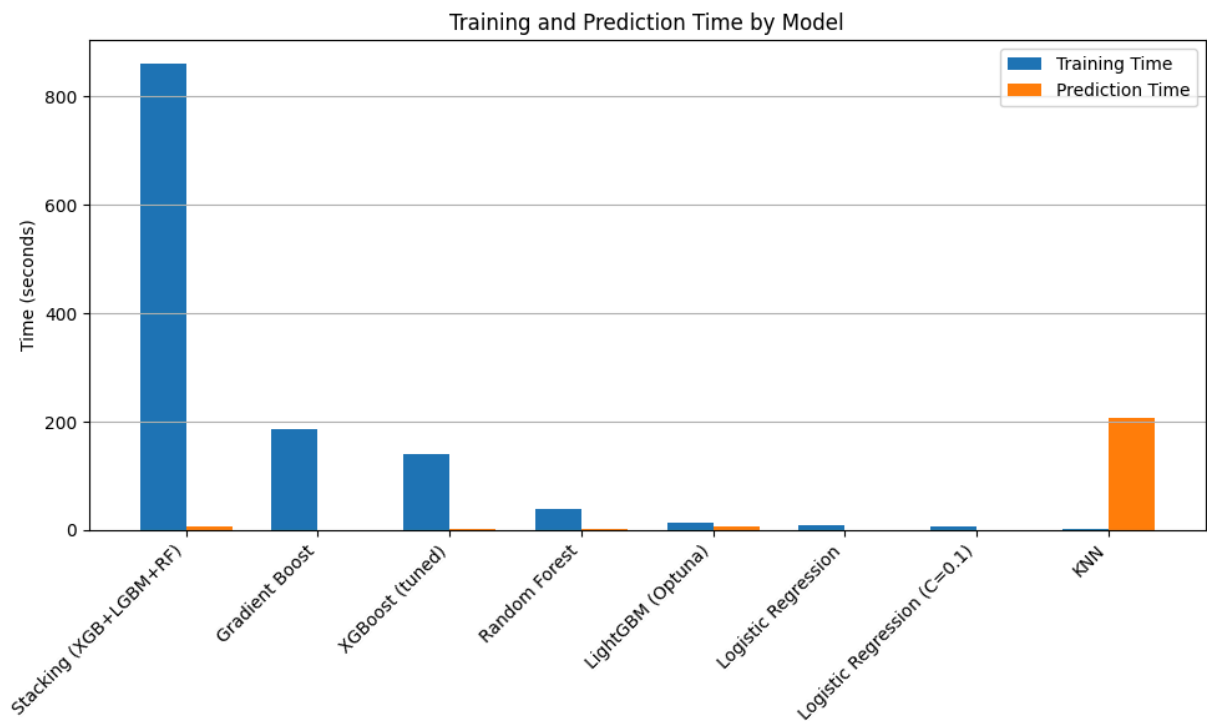
Accuracy: 0.8882
ROC AUC: 0.9559
Average Precision: 0.9369
Training Time: 860.6704 s
Prediction Time: 7.4147 s

```
In [ ]: boosting_model = XGBClassifier(  
    n_estimators=500,  
    learning_rate=0.05,  
    max_depth=6,  
    random_state=42,  
    n_jobs=-1,  
    verbosity=0,  
    eval_metric="auc",  
)  
  
boosting_model = evaluate_model(  
    "XGBoost (simple)",  
    boosting_model,  
    X_train_imputed, y_train_split,  
    X_val_imputed, y_val,  
)
```

```
In [ ]: # final results
plot_roc_curves(results)
plot_pr_curves(results, y_val)
plot_model_times(results)

results_df = results_table(results)
print(results_df)
```





	Model	Validation Accuracy	Validation AUC	\
0	LightGBM (Optuna)	0.889389	0.958206	
1	Stacking (XGB+LGBM+RF)	0.888185	0.955863	
2	XGBoost (tuned)	0.880542	0.953582	
3	Gradient Boost	0.853217	0.928237	
4	Random Forest	0.776789	0.917231	
5	Logistic Regression (C=0.1)	0.829451	0.910238	
6	Logistic Regression	0.829974	0.910186	
7	KNN	0.807098	0.880991	

	Validation AP	Training Time (s)	Prediction Time (s)
0	0.939083	13.253330	6.478119
1	0.936927	860.670369	7.414712
2	0.931588	140.651667	2.276553
3	0.896975	186.195160	0.509318
4	0.889925	38.080606	1.138932
5	0.868785	6.532904	0.128499
6	0.867986	9.983390	0.072164
7	0.832615	2.293277	206.558805

Model selection & tuning: I started with baseline models (logistic regression (plus adding regularization), KNN) and then moved to more complex tree-based models (Random Forests, Gradient Boosting, XGBoost, LightGBM). I used a validation split with ROC-AUC as the main metric and compared PR/ROC curves. For tuning, I applied manual tuning (e.g., depth, estimators), early stopping for XGBoost to find optimal trees, and Optuna for LightGBM to systematically search hyperparameters. I also tried a stacking ensemble combining top models. I had tried a grid search at first for Random Forest but due to computational limits, was unable to complete it, which is why I pivoted to trying gradient boosting models instead.

Best model & selection: LightGBM (Optuna) performed best (highest AUC ≈ 0.958 and strongest PR curve), closely followed by stacking and XGBoost. I selected it based on consistently highest validation AUC and better precision-recall tradeoff across thresholds, indicating stronger performance on imbalanced data.

In []:

1.3 Applying Model

Retrain models on full data and generate Kaggle submissions

```
In [ ]: X_train_full = add_features(X_train_ohe).copy()
X_test_full = add_features(X_test_ohe).copy()

imputer_full = SimpleImputer(strategy="median")

X_train_full_imputed = pd.DataFrame(
    imputer_full.fit_transform(X_train_full),
    columns=X_train_full.columns,
    index=X_train_full.index
)

X_test_full_imputed = pd.DataFrame(
    imputer_full.transform(X_test_full),
    columns=X_test_full.columns,
    index=X_test_full.index
)

scaler_full = StandardScaler()

X_train_full_scaled = pd.DataFrame(
    scaler_full.fit_transform(X_train_full_imputed),
    columns=X_train_full_imputed.columns,
    index=X_train_full_imputed.index
)

X_test_full_scaled = pd.DataFrame(
    scaler_full.transform(X_test_full_imputed),
    columns=X_test_full_imputed.columns,
    index=X_test_full_imputed.index
)

log_reg_final = LogisticRegression(max_iter=2000, random_state=42)
log_reg_final.fit(X_train_full_scaled, y_train)
log_reg_test_probs = log_reg_final.predict_proba(X_test_full_scaled)[: , 1]
create_submission(log_reg_test_probs, "/content/drive/MyDrive/Duke-
MIDS/ML/HW5/data/logistic_regression_submission.csv")

log_reg_l2_stronger_final = LogisticRegression(C=0.1, max_iter=2000,
random_state=42)
log_reg_l2_stronger_final.fit(X_train_full_scaled, y_train)
log_reg_l2_stronger_test_probs =
log_reg_l2_stronger_final.predict_proba(X_test_full_scaled)[: , 1]
create_submission(log_reg_l2_stronger_test_probs,
"/content/drive/MyDrive/Duke-
MIDS/ML/HW5/data/logistic_regression_c01_submission.csv")

knn_final = KNeighborsClassifier(n_neighbors=15)
knn_final.fit(X_train_full_scaled, y_train)
knn_test_probs = knn_final.predict_proba(X_test_full_scaled)[: , 1]
create_submission(knn_test_probs, "/content/drive/MyDrive/Duke-
MIDS/ML/HW5/data/knn_submission.csv")

gb_final = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    subsample=0.8,
    random_state=42
)
gb_final.fit(X_train_full_imputed, y_train)
gb_test_probs = gb_final.predict_proba(X_test_full_imputed)[: , 1]
create_submission(gb_test_probs, "/content/drive/MyDrive/Duke-
```



```
MIDS/ML/HW5/data/gradient_boost_submission.csv")

print("Saved submissions:")
print("./data/logistic_regression_submission.csv")
print("./data/logistic_regression_c01_submission.csv")
print("./data/knn_submission.csv")
print("./data/gradient_boost_submission.csv")
```

Saved submissions:

```
./data/logistic_regression_submission.csv
./data/logistic_regression_c01_submission.csv
./data/knn_submission.csv
./data/gradient_boost_submission.csv
```

```
In [ ]: rf_final = RandomForestClassifier(
    n_estimators=300,
    max_depth=12,
    min_samples_split=10,
    random_state=42,
    n_jobs=-1
)
rf_final.fit(X_train_full_imputed, y_train)
rf_test_probs = rf_final.predict_proba(X_test_full_imputed)[: , 1]
create_submission(rf_test_probs, "/content/drive/MyDrive/Duke-
MIDS/ML/HW5/data/random_forest_submission.csv")
```

```
In [ ]: xgb_final = XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    random_state=42,
    n_jobs=-1,
    verbosity=0,
    eval_metric="auc",
)
xgb_final.fit(X_train_full_imputed, y_train)
xgb_test_probs = xgb_final.predict_proba(X_test_full_imputed)[: , 1]
create_submission(xgb_test_probs,
    "/content/drive/MyDrive/Duke-
MIDS/ML/HW5/data/xgboost_simple_submission.csv")
print("Saved xgboost_simple_submission.csv")
```

New Model Submissions

```
In [ ]: # retrain XGBoost on all data and create submission
X_train_full_imp = add_features(X_train_ohe).copy()
X_test_full_imp = add_features(X_test_ohe).copy()

_imp = SimpleImputer(strategy="median")
X_train_full_imp = pd.DataFrame(
    _imp.fit_transform(X_train_full_imp),
    columns=X_train_full_imp.columns)
X_test_full_imp = pd.DataFrame(
    _imp.transform(X_test_full_imp),
    columns=X_test_full_imp.columns)

xgb_final = XGBClassifier(
    n_estimators=800, learning_rate=0.05, max_depth=6,
    subsample=0.8, colsample_bytree=0.8, min_child_weight=3,
    gamma=0.1, reg_alpha=0.1, reg_lambda=1.0,
    use_label_encoder=False, eval_metric="auc",
    random_state=42, n_jobs=-1, verbosity=0,
)
xgb_final.fit(X_train_full_imp, y_train)
xgb_test_probs = xgb_final.predict_proba(X_test_full_imp)[: , 1]
create_submission(xgb_test_probs,
    "/content/drive/MyDrive/Duke-MIDS/ML/HW5/data/xgboost_submission.csv")
print("Saved xgboost_submission.csv")
```

Saved xgboost_submission.csv

```
In [ ]: # retrain light GBM on all data and create submission
lgbm_final = lgb.LGBMClassifier(
    **study.best_params, random_state=42, n_jobs=-1, verbosity=-1,
)
lgbm_final.fit(X_train_full_imp, y_train)
lgbm_test_probs = lgbm_final.predict_proba(X_test_full_imp)[: , 1]
create_submission(lgbm_test_probs,
    "/content/drive/MyDrive/Duke-MIDS/ML/HW5/data/lightgbm_submission.csv")
print("Saved lightgbm_submission.csv")
```

Saved lightgbm_submission.csv

```
In [ ]: # retrain stacking on all data and create submission
stack_final = StackingClassifier(
    estimators=[
        ("xgb", XGBClassifier(
            n_estimators=800, learning_rate=0.05, max_depth=6,
            subsample=0.8, colsample_bytree=0.8, min_child_weight=3,
            gamma=0.1, reg_alpha=0.1, reg_lambda=1.0,
            use_label_encoder=False, eval_metric="auc",
            random_state=42, n_jobs=-1, verbosity=0,
        )),
        ("lgbm", lgb.LGBMClassifier(
            **study.best_params, random_state=42, n_jobs=-1, verbosity=-1,
        )),
        ("rf", RandomForestClassifier(
            n_estimators=300, max_depth=12, min_samples_split=10,
            random_state=42, n_jobs=-1,
        )),
    ],
    final_estimator=LR(C=1.0, max_iter=1000),
    cv=5,
    stack_method="predict_proba",
    n_jobs=-1,
)
stack_final.fit(X_train_full_imp, y_train)
stack_test_probs = stack_final.predict_proba(X_test_full_imp)[: , 1]
create_submission(stack_test_probs,
    "/content/drive/MyDrive/Duke-MIDS/ML/HW5/data/stacking_submission.csv")
print("Saved stacking_submission.csv")
```

Final Model Conclusion

The best performing model was Light GBM with test set AUC of 0.9621.

Exercise 2: Clustering

2.1: K Means

```
In [7]: #####
# Load the data
#####
import os
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs, make_moons

# Create / load the datasets:
n_samples = 1500
X0, _ = make_blobs(n_samples=n_samples, centers=2, n_features=2,
random_state=0)
X1, _ = make_blobs(n_samples=n_samples, centers=5, n_features=2,
random_state=0)

random_state = 170
X, y = make_blobs(n_samples=n_samples, random_state=random_state,
cluster_std=1.3)
transformation = [[0.6, -0.6], [-0.2, 0.8]]
X2 = np.dot(X, transformation)
X3, _ = make_blobs(n_samples=n_samples, cluster_std=[1.0, 2.5, 0.5],
random_state=random_state)
X4, _ = make_moons(n_samples=n_samples, noise=.12)

X = [X0, X1, X2, X3, X4]
# The datasets are X[i], where i ranges from 0 to 4

#####
# Code to plot clusters
#####
def plot_cluster(ax, data, cluster_assignments):
    '''Plot two-dimensional data clusters

    Parameters
    -----
    ax : matplotlib axis
        Axis to plot on
    data : list or numpy array of size [N x 2]
        Clustered data
    cluster_assignments : list or numpy array [N]
        Cluster assignments for each point in data

    ...

    clusters = np.unique(cluster_assignments)
    n_clusters = len(clusters)
    for ca in clusters:
        kwargs = {}
        if ca == -1:
            # if samples are not assigned to a cluster (have a cluster
            assignment of -1, color them gray)
            kwargs = {'color':'gray'}
            n_clusters = n_clusters - 1
        ax.scatter(data[cluster_assignments==ca, 0],
data[cluster_assignments==ca, 1],s=5,alpha=0.5, **kwargs)
        ax.set_xlabel('feature 1')
        ax.set_ylabel('feature 2')
        ax.set_title(f'No. Clusters = {n_clusters}')
        ax.axis('equal')

# scatter plot each dataset
fig, axes = plt.subplots(1, 5, figsize=(22, 4))
```

```

for i, data in enumerate(X):
    axes[i].scatter(data[:, 0], data[:, 1], s=5, alpha=0.5)
    axes[i].set_title(f"Dataset {i}")
    axes[i].set_xlabel("feature 1")
    axes[i].set_ylabel("feature 2")
    axes[i].axis("equal")
plt.tight_layout()
plt.show()

from sklearn.cluster import KMeans, DBSCAN, SpectralClustering
from sklearn.metrics import silhouette_score

k_values = range(1, 11)
inertias = {}

fig, axes = plt.subplots(1, 5, figsize=(22, 4))
for i, data in enumerate(X):
    dataset_inertias = []
    for k in k_values:
        km = KMeans(n_clusters=k, random_state=0, n_init=10)
        km.fit(data)
        dataset_inertias.append(km.inertia_)
    inertias[i] = dataset_inertias

    axes[i].plot(list(k_values), dataset_inertias, marker='o')
    axes[i].set_title(f"Dataset {i} Elbow Curve")
    axes[i].set_xlabel("k")
    axes[i].set_ylabel("inertia")
plt.tight_layout()
plt.show()

from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans

k_values = range(2, 11) # silhouette not defined for k=1
silhouette_scores = {}

fig, axes = plt.subplots(1, 5, figsize=(22, 4))

for i, data in enumerate(X):
    scores = []

    for k in k_values:
        km = KMeans(n_clusters=k, random_state=0, n_init=10)
        labels = km.fit_predict(data)
        score = silhouette_score(data, labels)
        scores.append(score)

    silhouette_scores[i] = scores

    axes[i].plot(list(k_values), scores, marker='o')
    axes[i].set_title(f"Dataset {i} Silhouette")
    axes[i].set_xlabel("k")
    axes[i].set_ylabel("score")

plt.tight_layout()
plt.show()

best_k_silhouette = {}

for i in range(len(X)):
    scores = silhouette_scores[i]
    best_k = list(k_values)[np.argmax(scores)]
    best_k_silhouette[i] = best_k

print(best_k_silhouette)

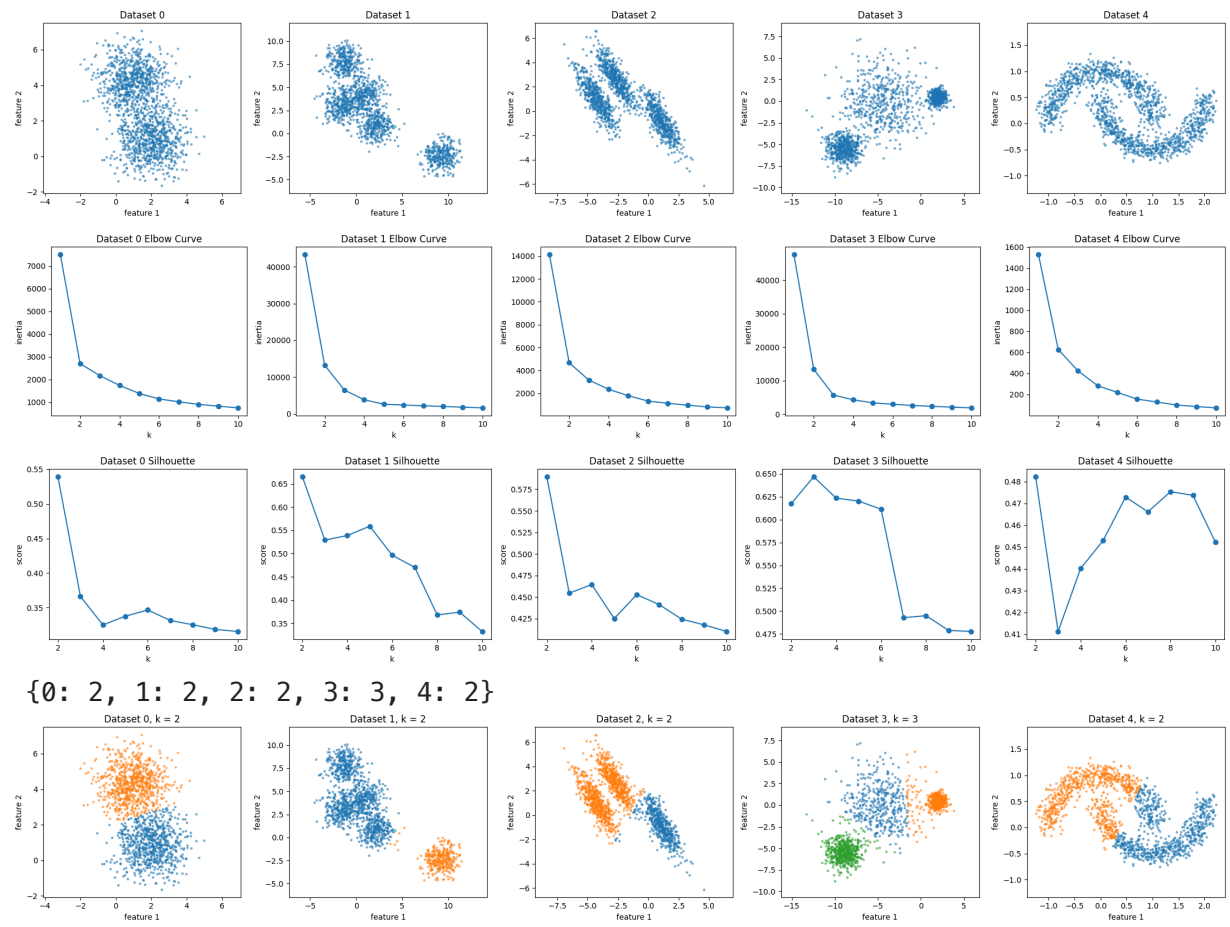
fig, axes = plt.subplots(1, 5, figsize=(22, 4))

for i, data in enumerate(X):
    k = best_k_silhouette[i]
    km = KMeans(n_clusters=k, random_state=0, n_init=10)
    labels = km.fit_predict(data)

```

```
plot_cluster(axes[i], data, labels)
axes[i].set_title(f"Dataset {i}, k = {k}")

plt.tight_layout()
plt.show()
```



Interpretation:

From the elbow curves for each dataset (dataset 0, 1, 2, 4), the elbow appears at k=2 (except for dataset 3 which performed better with k=3 clusters), which indicates this is where the inertia begins to have diminishing returns as we increase number of clusters, and this value of k could be a good balance.

2.2: DBSCAN

```
In [8]: # change eps and min_samples parameters to get as close as you can to
        # having the same number of clusters as your choices with K-means
        from sklearn.cluster import DBSCAN

        eps_values = np.linspace(0.1, 1.5, 15)
        min_samples_values = [3, 5, 10, 15]

        best_dbscan_params = {}
        best_dbscan_labels = {}

        # search for the best dbscan params to get as close to number of clusters
        # as above
        for i, data in enumerate(X):

            # number of cluster choices for each dataset from kmeans
            target_k = best_k_silhouette[i]
            best_diff = float("inf")
            best_params = None
            best_labels = None

            for eps in eps_values:
                for min_samples in min_samples_values:
                    db = DBSCAN(eps=eps, min_samples=min_samples)
                    labels = db.fit_predict(data)

                    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
                    diff = abs(n_clusters - target_k)

                    if diff < best_diff:
                        best_diff = diff
                        best_params = (eps, min_samples, n_clusters)
                        best_labels = labels

            best_dbscan_params[i] = best_params
            best_dbscan_labels[i] = best_labels

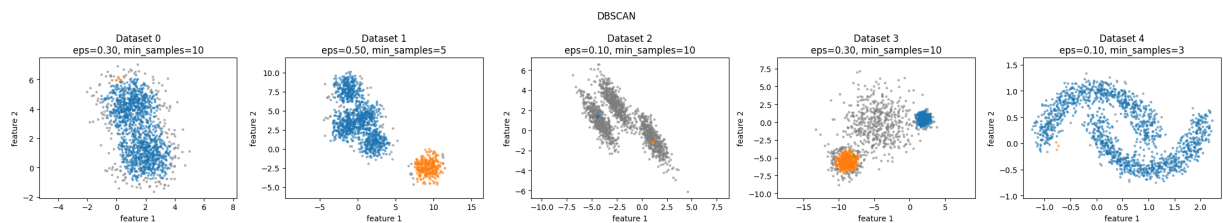
        for i in range(len(X)):
            eps, min_samples, n_clusters = best_dbscan_params[i]
            print(f"Dataset {i}: target k = {best_k_silhouette[i]}, eps =
            {eps:.2f}, min_samples = {min_samples}, DBSCAN clusters = {n_clusters}")
```

Dataset 0: target k = 2, eps = 0.30, min_samples = 10, DBSCAN clusters = 2
Dataset 1: target k = 2, eps = 0.50, min_samples = 5, DBSCAN clusters = 2
Dataset 2: target k = 2, eps = 0.10, min_samples = 10, DBSCAN clusters = 2
Dataset 3: target k = 3, eps = 0.30, min_samples = 10, DBSCAN clusters = 3
Dataset 4: target k = 2, eps = 0.10, min_samples = 3, DBSCAN clusters = 2

```
In [9]: fig, axes = plt.subplots(1, 5, figsize=(22, 4))
        fig.suptitle("DBSCAN")

        for i, data in enumerate(X):
            plot_cluster(axes[i], data, best_dbscan_labels[i])
            eps, min_samples, n_clusters = best_dbscan_params[i]
            axes[i].set_title(f"Dataset {i}\neps={eps:.2f}, min_samples=
            {min_samples}")

        plt.tight_layout()
        plt.show()
```



2.3: Spectral Clustering

```
In [10]: #Apply Spectral Clustering. Select the same number of clusters as selected
         #by k-means.
         from sklearn.cluster import SpectralClustering

         sc_labels = {}

         for i, data in enumerate(X):

             # number of cluster choices for each dataset from kmeans
             target_k = best_k_silhouette[i]

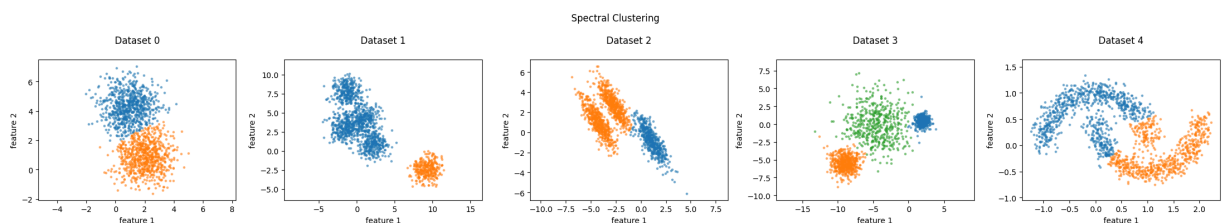
             sc = SpectralClustering(
                 n_clusters=target_k,
                 affinity='nearest_neighbors',
                 n_neighbors=10,
                 random_state=42
             )

             labels = sc.fit_predict(data)
             sc_labels[i] = labels

         fig, axes = plt.subplots(1, 5, figsize=(22, 4))
         fig.suptitle("Spectral Clustering")

         for i, data in enumerate(X):
             plot_cluster(axes[i], data, sc_labels[i])
             axes[i].set_title(f"Dataset {i}\n")

         plt.tight_layout()
         plt.show()
```



Interpretation:

On the more simple structured blob datasets (0, 1, and 3), KMeans worked well without much tuning needed beyond choosing k . However, on dataset 2, KMeans did not perform as well with the elongated clusters in dataset 2 and on the moons dataset 4. DBSCAN worked very well on dataset 4 as it could capture those non-linear shapes, but it was inconsistent on the other datasets, especially dataset 2 and 3. And DBSCAN required the most tuning for ϵ and min_samples , which are sensitive. Spectral Clustering seemed to be consistent across all datasets, as it performed similar to Kmeans on the simple data while also doing well on the non-linear data structures as well. KMeans is the easiest but least flexible, while DBSCAN is the hardest to tune and good for complex shapes, but overall, Spectral Clustering has the best balance.

Exercise 3 - Dimensionality Reduction

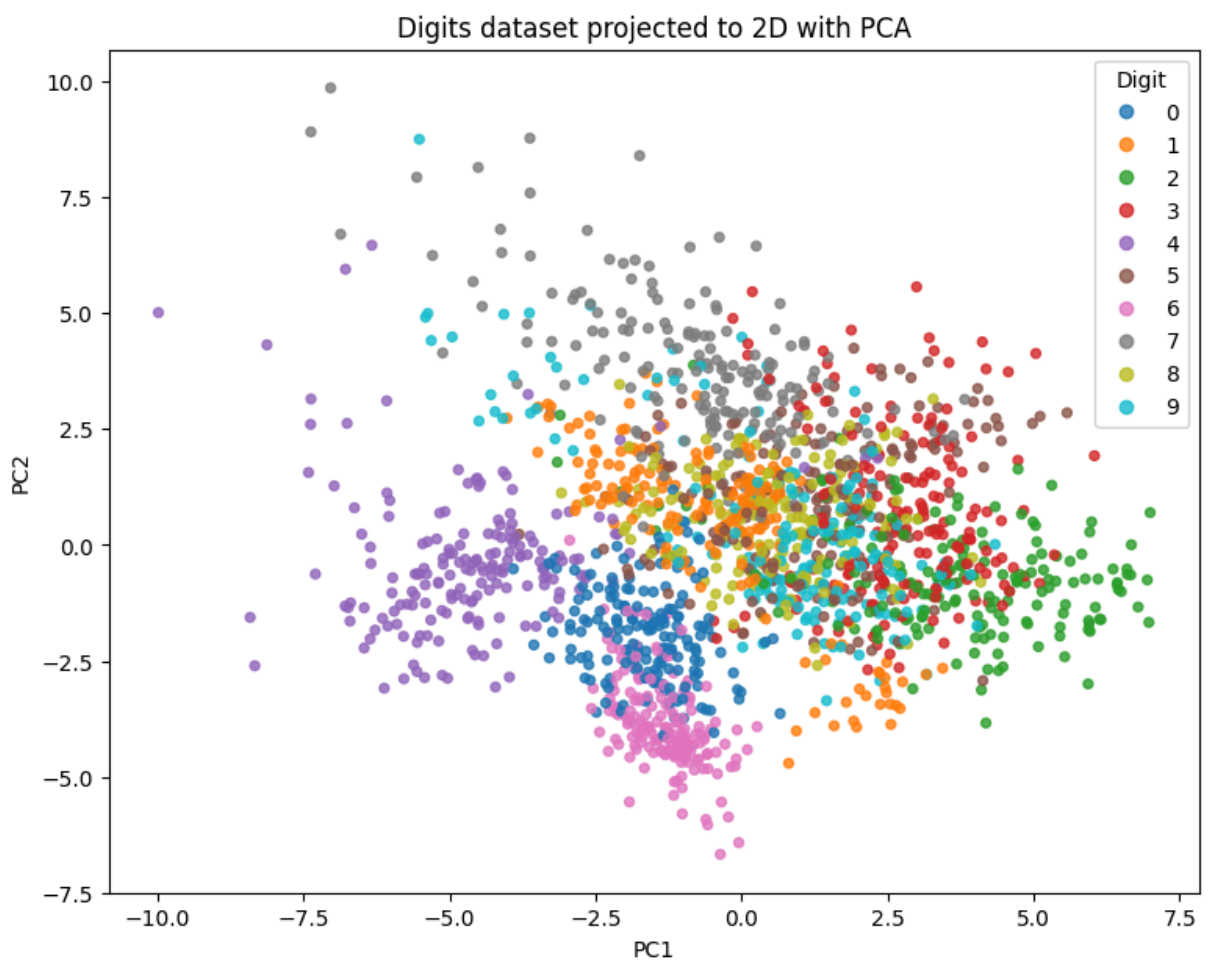
```
In [11]: #####  
# Load the data  
#####  
from sklearn import datasets  
from sklearn.decomposition import PCA  
from sklearn.manifold import TSNE  
  
# load dataset  
digits = datasets.load_digits()  
n_sample = digits.target.shape[0]  
n_feature = digits.images.shape[1] * digits.images.shape[2]  
X_digits = np.zeros((n_sample, n_feature))  
for i in range(n_sample):  
    X_digits[i, :] = digits.images[i, :, :].flatten()  
y_digits = digits.target
```

3.1: PCA

```
In [12]: from sklearn.preprocessing import StandardScaler
# standardize before PCA / t-SNE
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_digits)

# pca to 2d
pca_2d = PCA(n_components=2, random_state=42)
X_pca_2d = pca_2d.fit_transform(X_scaled)

plt.figure(figsize=(9, 7))
scatter = plt.scatter(X_pca_2d[:, 0], X_pca_2d[:, 1], c=y_digits,
                      cmap="tab10", s=18, alpha=0.8)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("Digits dataset projected to 2D with PCA")
plt.legend(*scatter.legend_elements(), title="Digit")
plt.show()
```



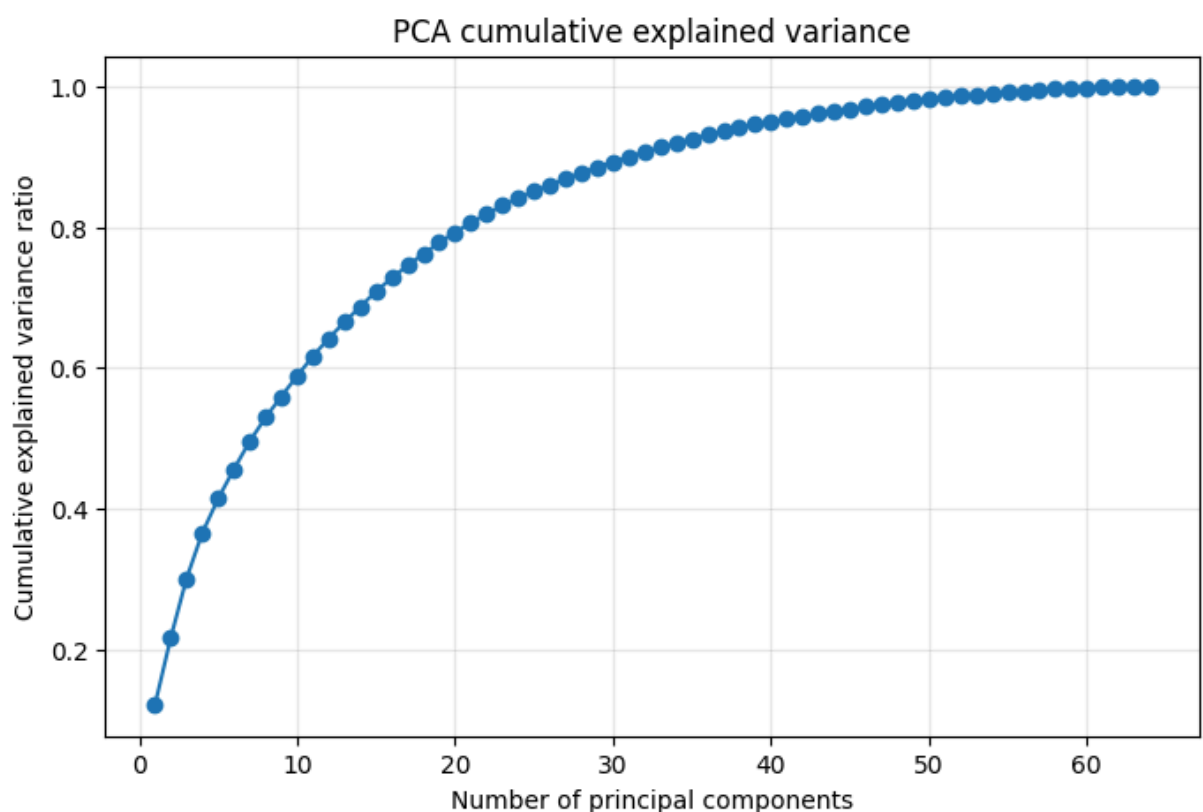
I standardized the digits data before PCA because PCA is variance-based, so features on larger scales can dominate the principal components. After reducing the data to 2 dimensions, the PCA plot shows some visible grouping by digit, but several classes still overlap because projecting to only two linear components loses a large amount of information.

3.2: Plot Variance Explained

```
In [13]: # cumulative explained variance
pca_full = PCA().fit(X_scaled)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)

plt.figure(figsize=(8, 5))
plt.plot(range(1, len(cum_var) + 1), cum_var, marker="o")
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance ratio")
plt.title("PCA cumulative explained variance")
plt.grid(True, alpha=0.3)
plt.show()

unexplained_2pc = 1 - cum_var[1]
print("Fraction unexplained by first 2 PCs:", unexplained_2pc)
```



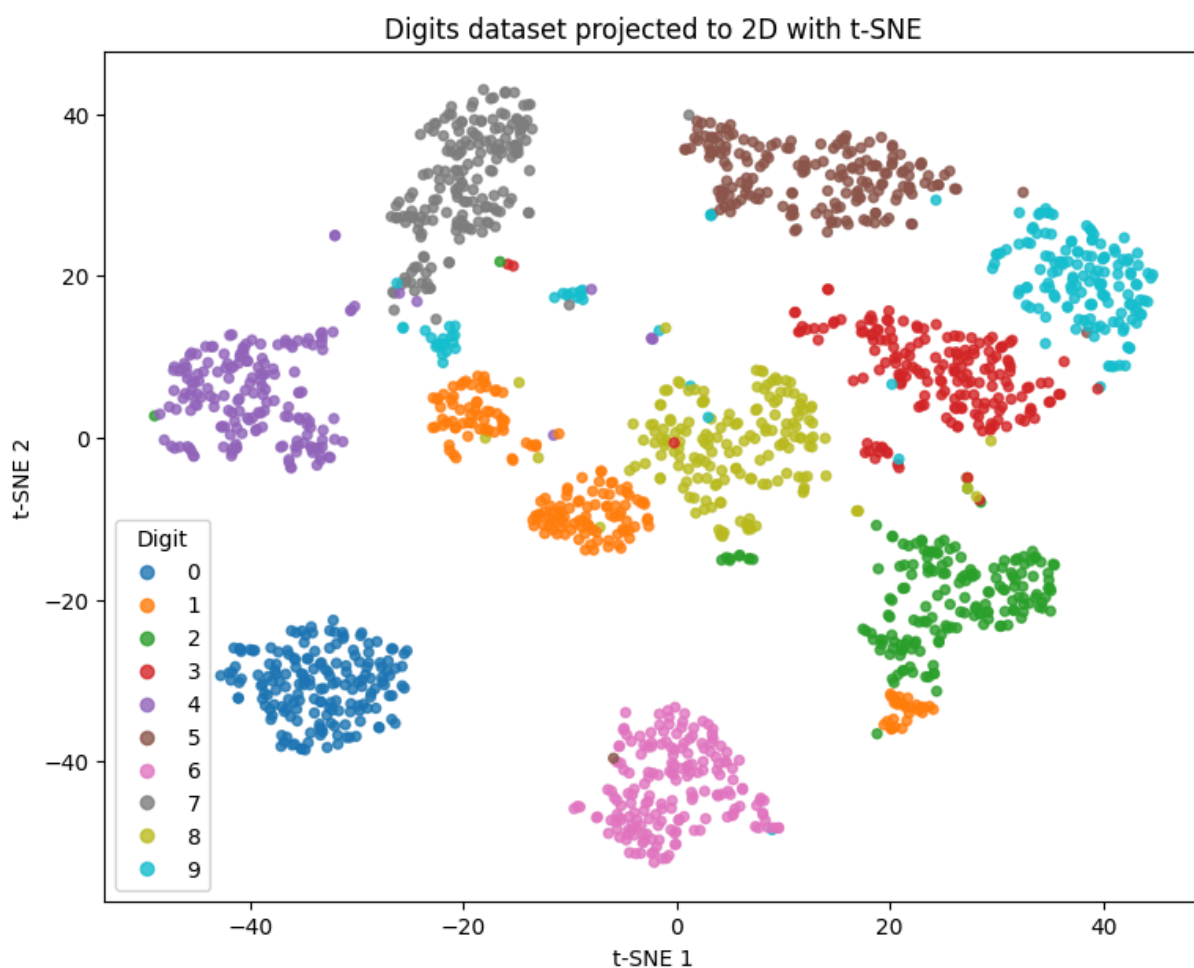
Fraction unexplained by first 2 PCs: 0.7840502949916721

The cumulative explained variance plot shows how much total variance is captured as more principal components are added. The fraction of variance unexplained by the first two principal components is $1 - \text{cumulative variance at 2 PCs}$, which here is the printed value above (0.784). Since a large fraction of the variance is unexplained, the 2D PCA visualization cannot perfectly separate the digits, so overlap between classes is expected.

3.3: t-SNE

```
In [14]: # t-SNE to 2D
tsne = TSNE(n_components=2, perplexity=30, random_state=42, init="pca",
            learning_rate="auto")
X_tsne_2d = tsne.fit_transform(X_scaled)

plt.figure(figsize=(9, 7))
scatter = plt.scatter(X_tsne_2d[:, 0], X_tsne_2d[:, 1], c=y_digits,
                    cmap="tab10", s=18, alpha=0.8)
plt.xlabel("t-SNE 1")
plt.ylabel("t-SNE 2")
plt.title("Digits dataset projected to 2D with t-SNE")
plt.legend(*scatter.legend_elements(), title="Digit")
plt.show()
```



t-SNE gives a 2D embedding that is usually much better at separating the digit classes visually because it is able to preserve local neighborhood structure instead of maximizing global variance. In the t-SNE plot, the classes tend to form tighter and more distinct groups than in PCA.

3.4: Conclusion

TSNE clusters the data better than PCA because the dataset has non linear structure so PCA can't fully capture this with only 2 linear components. PCA is faster and simpler however, with more interpretable explained variance while TSNE is good for visualization but requires more computation. That is why in scikit learn, TSNE usually has `fit_transform` and not a separate transform, as it does not provide a true transform for new points. The embedding comes from the full dataset so adding new points isn't added in the same way that PCA does.

```
In [ ]: # !pip install -q nbconvert
        # !jupyter nbconvert --to html HW5.ipynb --
        TemplateExporter.exclude_input_prompt=True
```

```
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
[NbConvertApp] WARNING | pattern 'hw4.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
    to various other formats.
```

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options

=====

The options below are convenience aliases to configurable class-options, as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

<cmd> --help-all

--debug

set log level to logging.DEBUG (maximize logging output)

Equivalent to: [--Application.log_level=10]

--show-config

Show the application's configuration (human-readable format)

Equivalent to: [--Application.show_config=True]

--show-config-json

Show the application's configuration (json format)

Equivalent to: [--Application.show_config_json=True]

--generate-config

generate default config file

Equivalent to: [--JupyterApp.generate_config=True]

-y

Answer yes to any questions instead of prompting.

Equivalent to: [--JupyterApp.answer_yes=True]

--execute

Execute the notebook prior to export.

Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors

Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was specified, to o.

Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin

read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'

Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout

Write notebook output to stdout instead of files.

Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace

Run nbconvert in place, overwriting the existing notebook (only relevant when converting to notebook format)

Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory=]

--clear-output

Clear output of current file and save in place, overwriting the existing notebook.

Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --ClearOutputPreprocessor.enabled=True]

--coalesce-streams

Coalesce consecutive stdout and stderr outputs into one stream (within each cell).

Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --CoalesceStreamsPreprocessor.enabled=True]

--no-prompt

Exclude input and output prompts from converted document.

Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.exclude_output_prompt=True]

--no-input

Exclude input cells and output prompts from converted document.

This mode is ideal for generating code-free reports.

Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporter.exclude_input=True --TemplateExporter.exclude_input_prompt=True]

--allow-chromium-download

Whether to allow downloading chromium if no suitable version is found on the system.
Equivalent to: [--WebPDFExporter.allow_chromium_download=True]

--disable-chromium-sandbox
Disable chromium security sandbox when converting to PDF..
Equivalent to: [--WebPDFExporter.disable_sandbox=True]

--show-input
Shows code input. This flag is only useful for dejavu users.
Equivalent to: [--TemplateExporter.exclude_input=False]

--embed-images
Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.
Equivalent to: [--HTMLExporter.embed_images=True]

--sanitize-html
Whether the HTML in Markdown cells and cell outputs should be sanitized..
Equivalent to: [--HTMLExporter.sanitize_html=True]

--log-level=<Enum>
Set the log level by value or name.
Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']
Default: 30
Equivalent to: [--Application.log_level]

--config=<Unicode>
Full path of a config file.
Default: ''
Equivalent to: [--JupyterApp.config_file]

--to=<Unicode>
The export format to be used, either one of the built-in formats ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'] or a dotted object name that represents the import path for an ``Exporter`` class
Default: ''
Equivalent to: [--NbConvertApp.export_format]

--template=<Unicode>
Name of the template to use
Default: ''
Equivalent to: [--TemplateExporter.template_name]

--template-file=<Unicode>
Name of the template file to use
Default: None
Equivalent to: [--TemplateExporter.template_file]

--theme=<Unicode>
Template specific theme(e.g. the name of a JupyterLab CSS theme distributed as prebuilt extension for the lab template)
Default: 'light'
Equivalent to: [--HTMLExporter.theme]

--sanitize_html=<Bool>
Whether the HTML in Markdown cells and cell outputs should be sanitized. This should be set to True by nbviewer or similar tools.
Default: False
Equivalent to: [--HTMLExporter.sanitize_html]

--writer=<DottedObjectName>
Writer class used to write the results of the conversion
Default: 'FilesWriter'
Equivalent to: [--NbConvertApp.writer_class]

--post=<DottedOrNone>
PostProcessor class used to write the results of the conversion
Default: ''
Equivalent to: [--NbConvertApp.postprocessor_class]

--output=<Unicode>
Overwrite base name use for output files.
Supports pattern replacements '{notebook_name}'.
Default: '{notebook_name}'
Equivalent to: [--NbConvertApp.output_base]

--output-dir=<Unicode>
Directory to write output(s) to. Defaults to output to the directory of each notebook

k. To recover previous default behaviour (outputting to the current working directory) use `.` as the flag value.

Default: ''
 Equivalent to: `[--FilesWriter.build_directory]`
`--reveal-prefix=<Unicode>`
 The URL prefix for reveal.js (version 3.x).
 This defaults to the reveal CDN, but can be any url pointing to a copy of reveal.js.
 For speaker notes to work, this must be a relative path to a local copy of reveal.js: e.g., "reveal.js".
 If a relative path is given, it must be a subdirectory of the current directory (from which the server is run).
 See the usage documentation
 (<https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js>
`--html-slideshow`)
 for more details.
 Default: ''
 Equivalent to: `[--SlidesExporter.reveal_url_prefix]`
`--nbformat=<Enum>`
 The nbformat version to write.
 Use this to downgrade notebooks.
 Choices: any of [1, 2, 3, 4]
 Default: 4
 Equivalent to: `[--NotebookExporter.nbformat_version]`

Examples

The simplest way to use nbconvert is

```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX includes 'base', 'article' and 'report'. HTML includes 'basic', 'lab' and 'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of different ways:

```
> jupyter nbconvert notebook*.ipynb
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```

```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use `--help-all`.

In []:

In []: